FMX Academy

INFORMAÇÕES			
MÓDULO		**AULA**	**INSTRUTOR**
07	**ASP.NET Core Web API – Avançado - Padrões de Projeto, Arquitetura Limpa e Alta Performance**	**04 - Validação de Entrada com FluentValidation e Generics**	**Guilherme Paracatu**

1. Teoria e Conceitos

O Conceito de "Fail-Fast" (Falhe Rápido)
No desenvolvimento de software de alta performance, seguimos o princípio **Fail-Fast**. Ele diz que, se uma requisição contém dados inválidos, ela deve ser interrompida o mais cedo possível.

Por que isso é importante? Sem a validação de entrada, o comando inválido viajaria pela API, passaria pelo Service Bus, chegaria ao Handler, abriria uma conexão com o banco de dados e só então falharia. Com o FluentValidation, barramos o erro na "porta de entrada", economizando recursos preciosos do servidor.

A Anatomia da Validação: Fluent vs. Imperativa

- **Validação Imperativa (O jeito antigo):** Você escreve vários if/else dentro do seu código de negócio. Isso torna o código difícil de ler e de manter.

- **Validação Fluida (O jeito moderno):** Você declara regras de forma legível, quase como uma frase em inglês: *"Rule for Placa: Not Empty, Length 7"*. Isso é o que chamamos de **DSL (Domain Specific Language)** interna.

O que é o FluentValidation?

O **FluentValidation** é uma biblioteca popular para .NET que utiliza uma interface fluida (encadeamento de métodos) para construir regras de validação fortemente tipadas. No ecossistema da nossa arquitetura, ele atua como a **primeira linha de defesa** da nossa aplicação.

Por que não usar Data Annotations ([Required], [StringLength])?
Embora os atributos (Attributes) sejam o padrão nativo do .NET, eles apresentam limitações em projetos profissionais:

1. **Poluição do Modelo:** Suas classes de comando (Commands) ficam cheias de "sujeira" visual.

2. **Dificuldade em Regras Complexas:** Validar se um campo depende de outro usando

atributos é extremamente trabalhoso.

3. **Violação do Single Responsibility Principle (SRP):** A classe de dados não deveria conhecer as regras de como ela própria é validada.

A Diferença entre Validação de Entrada e Validação de Domínio
Este é o ponto onde muitos desenvolvedores se confundem:

- **Validação de Entrada (FluentValidation):** Verifica o "formato". Ex: "A placa tem 7 caracteres?", "O nome está vazio?". Ela acontece na camada de **Aplicação**, antes do comando chegar ao Handler.

- **Validação de Domínio (Exceptions no Objeto):** Verifica o "estado e regra de negócio". Ex: "Este veículo pode ser desativado tendo multas em aberto?". Ela acontece na camada de **Domínio**, dentro da Entidade.

Benefícios no Dia a Dia:
- **Mensagens Personalizadas:** Controle total sobre o que o usuário final vai ler.
- **Reaproveitamento:** Você define a regra uma vez e ela vale para qualquer lugar que use aquele Command.
- **Testabilidade:** É possível criar testes unitários para as validações sem precisar de banco de dados ou API ligada.

2. Guia Prático Passo a Passo

Passo 1: Instalação dos Pacotes (NuGet)
Para que a arquitetura funcione, precisamos instalar as bibliotecas nos projetos corretos. Abram o terminal nas respectivas pastas:

- **No projeto ITB.Application:** (Onde ficam as regras)
  dotnet add ITB.Application package FluentValidation

- **No projeto ITB.IoC:** (Onde fica a injeção de dependência)
  dotnet add ITB.IoC package FluentValidation.DependencyInjectionExtensions
  dotnet add ITB.IoC package FluentValidation.AspNetCore

Passo 2: Criando a Classe de Validação
Este é o arquivo onde as regras são escritas. Ele deve ser criado na camada de **Aplicação.**

Local: ITB.Application/Validations/AdicionarVeiculoValidation.cs
Código C#
```
using FluentValidation;
using ITB.Application.Commands;

namespace ITB.Application.Validations;
```

```
public class AdicionarVeiculoValidation : AbstractValidator<AdicionarVeiculoCommand>
{
    public AdicionarVeiculoValidation()
    {
        RuleFor(v => v.Placa)
            .NotEmpty().WithMessage("A placa é de preenchimento obrigatório.")
            .Length(7).WithMessage("A placa deve possuir exatamente 7 caracteres.")
            .Matches(@"^[A-Z]{3}[0-9][A-Z0-9][0-9]{2}$")
            .WithMessage("A placa deve seguir o padrão Mercosul (Ex: ABC1D23).");

        RuleFor(v => v.Ano)
            .InclusiveBetween(1900, DateTime.Now.Year + 1)
            .WithMessage("O ano informado é inválido.");

        RuleFor(v => v.ModeloId)
            .GreaterThan(0).WithMessage("Por favor, selecione um modelo de veículo
válido.");
    }
}
```

Passo 3: Configuração no Projeto de IoC
Vamos centralizar o registro para que a API não precise conhecer os validadores individualmente.

Local: ITB.IoC/DependencyInjection.cs
Código C#

```
using FluentValidation;
using FluentValidation.AspNetCore;
using ITB.Application.Validations;
using Microsoft.Extensions.DependencyInjection;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(this IServiceCollection
services)
    {
        // 1. Registra todos os validadores do Assembly de Application automaticamente
        services.AddValidatorsFromAssemblyContaining<AdicionarVeiculoValidation>();

        // 2. Habilita a validação automática para o ASP.NET
        services.AddFluentValidationAutoValidation();

        // ... demais registros (Repositories, Context, etc)
        return services;
    }
}
```

Passo 4: Criando o Filtro de Validação (API)
O "Filtro" intercepta a requisição antes de chegar na Controller. Se o ModelState estiver inválido (preenchido pelo FluentValidation), ele já retorna o erro.

Local: ITB.Api/Filters/ValidationFilter.cs
Código C#

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace ITB.Api.Filters;

public class ValidationFilter : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(ActionExecutingContext context,
ActionExecutionDelegate next)
    {
        if (!context.ModelState.IsValid)
        {
            var mensagensDeErro = context.ModelState
                .Where(x => x.Value?.Errors.Count > 0)
                .ToDictionary(
                    kvp => kvp.Key,
                    kvp => kvp.Value?.Errors.Select(e => e.ErrorMessage).ToArray()
                );

            context.Result = new BadRequestObjectResult(new {
                mensagem = "Dados inválidos",
                erros = mensagensDeErro
            });
            return;
        }

        await next();
    }
}
```

Passo 5: Ativando o Filtro Globalmente
O último passo é dizer ao .NET para usar nosso filtro em todas as rotas.

Local: ITB.Api/Program.cs
Código C#

```
builder.Services.AddControllers(options =>
{
    // Adiciona o filtro globalmente
    options.Filters.Add<ValidationFilter>();
});
```

Passo 6: Controller Impecável
Agora a Controller foca apenas no que importa: orquestrar o envio do comando.

Local: ITB.Api/Controllers/VeiculoController.cs
Código C#

```
[HttpPost]
public async Task<IActionResult> Post([FromBody] AdicionarVeiculoCommand command)
{
    // A validação já ocorreu no Filtro antes de chegar aqui!
    await _bus.EnviarComando(command);
    return Ok(new { mensagem = "Operação realizada com sucesso!" });
}
```

📚 Referência Rápida: Regras mais Utilizadas

O **FluentValidation** possui dezenas de validadores embutidos. Abaixo estão os mais comuns no dia a dia do desenvolvimento:

Regra	Descrição	Exemplo de Uso
NotEmpty()	Garante que o campo não seja nulo, vazio ou espaços em branco.	.RuleFor(x => x.Nome).NotEmpty();
Length(min, max)	Define um tamanho exato ou intervalo de caracteres para strings.	.Length(7) ou .Length(2, 50)
InclusiveBetween()	Verifica se um número ou data está dentro de um intervalo.	.InclusiveBetween(1900, 2026)
Matches()	Valida o campo contra uma Expressão Regular (Regex).	.Matches(@"^[0-9]{5}-[0-9]{3}$")
EmailAddress()	Valida se a string possui um formato de e-mail válido.	.RuleFor(x => x.Email).EmailAddress();
GreaterThan()	Verifica se o valor é maior que um limite (ex: IDs > 0).	.GreaterThan(0)
Equal()	Compara o valor com outra propriedade ou constante.	.Equal(x => x.ConfirmarSenha)
Must()	Permite criar uma lógica customizada (validação manual).	.Must(x => x.Id % 2 == 0)

🔗 Links Úteis para Estudo

Para aprofundar os conhecimentos e resolver cenários complexos:

1. Documentação Oficial do FluentValidation

O melhor lugar para entender regras avançadas, como validações em cascata e coleções.

🌐 docs.fluentvalidation.net

2. Regex101 (O melhor amigo da validação)
As Expressões Regulares (Regex) são fundamentais para validar Placas, CPFs, CEPs e Telefones. Este site permite testar suas Regex em tempo real antes de colocá-las no código C#.
🌐 regex101.com
Dica: Ao acessar, selecione o sabor ".NET (C#)" na barra lateral esquerda.

3. FluentValidation no GitHub
Útil para acompanhar as discussões da comunidade e ver exemplos de implementação em larga escala.
🌐 github.com/FluentValidation/FluentValidation

🗣 **Dica:**
"Pessoal, não tentem decorar todas as regras. Decorem o que é possível fazer e consultem a documentação sempre que surgir um cenário novo. O importante é saber onde procurar!"

3. Generics e Reutilização de Código

1. O que são Generics (Tipos Genéricos)?

Em C#, **Generics** permitem que você defina classes, interfaces e métodos com **"espaços reservados" (placeholders)** para os tipos de dados.

Em vez de escrever uma classe que funciona apenas para Veiculo e outra igualzinha para Marca, você escreve uma única classe que funciona para um tipo T. Esse T será definido apenas no momento em que o código for executado.

A Analogia da Fábrica de Caixas:

Imagine uma fábrica que produz caixas de papelão.

- **Sem Generics:** A fábrica precisa de uma máquina específica para criar "Caixa de Sapatos", outra máquina para "Caixa de Livros" e outra para "Caixa de Brinquedos".

- **Com Generics:** A fábrica tem uma única máquina chamada "Criador de Caixa <T>". Quando você liga a máquina, você diz: *"Hoje, o T será Sapato"*. A máquina se ajusta e produz caixas para sapatos. Amanhã, você diz *"Hoje, o T será Livro"*.

2. Por que usar Generics? (As 3 Dores que ele resolve)

1. **Reutilização de Código (DRY - Don't Repeat Yourself):**

   Sem Generics, você teria que copiar e colar a lógica de CRUD (Create, Read, Update, Delete) para cada nova tabela do banco. Com Generics, você escreve o CRUD uma única vez e ele serve para todo o sistema.

2. **Type Safety (Segurança de Tipo):**

   Antigamente, usava-se o tipo object para tornar as coisas genéricas. O problema é que object aceita tudo. Você poderia acidentalmente tentar salvar uma Marca na tabela de Veiculos e o compilador não reclamaria (o erro só explodiria na cara do cliente). Com Generics, se você definir Repositorio<Veiculo>, o compilador **garante** que só entram veículos ali.

3. **Desempenho (Boxing e Unboxing):**

   Generics são resolvidos em tempo de compilação. Isso evita que o .NET precise ficar convertendo tipos na memória (o custoso processo de Boxing/Unboxing), tornando o sistema mais leve e rápido.

3. As Restrições (Constraints - A Cláusula where)

O maior poder dos Generics vem da capacidade de limitar o que o T pode ser. Não queremos que alguém crie um Repositorio<int>, certo? Repositórios são para classes!

Para isso usamos a palavra-chave **where**:

Restrição	**Código**	**O que significa?**
Reference Type	where T : class	O tipo T deve ser obrigatoriamente uma Classe (não pode ser int, bool, etc).
Constructor	where T : new()	O tipo T deve ter um construtor público vazio (permite fazer new T()).
Herança	where T : EntidadeBase	O tipo T deve ser filho da classe EntidadeBase.
Interface	where T : IEntidade	O tipo T deve implementar a interface IEntidade.

4. Prática - O Padrão Generic Repository

Agora vamos aplicar isso na nossa arquitetura. Atualmente, VeiculoRepository e MarcaRepository têm códigos quase idênticos para Adicionar, Remover e ObterPorId. Vamos unificar isso.

Passo 1: A Interface Genérica (IRepositoryBase)

Primeiro, definimos o contrato. Note o uso de <TEntity> e a restrição where TEntity : class.

Local: ITB.Domain/Interfaces/IRepositoryBase.cs

Código C#

```
using System.Collections.Generic;
using System.Threading.Tasks;

namespace ITB.Domain.Interfaces;

// Definimos que esta interface aceita um tipo genérico TEntity
// E obrigamos que esse TEntity seja uma classe.
public interface IRepositoryBase<TEntity> where TEntity : class
{
    Task<TEntity?> ObterPorId(int id);
    Task<IEnumerable<TEntity>> ObterTodos();
    Task Adicionar(TEntity entity);
    void Remover(TEntity entity);
}
```

Passo 2: A Implementação Genérica (RepositoryBase)

Aqui acontece a mágica do Entity Framework Core. O método Set<T>() do DbContext permite acessar a tabela correspondente ao tipo T dinamicamente.

Local: ITB.Infrastructure/Repositories/RepositoryBase.cs

Código C#

```
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ITB.Domain.Interfaces;
using ITB.Infrastructure.Persistence;

namespace ITB.Infrastructure.Repositories;

public abstract class RepositoryBase<TEntity> : IRepositoryBase<TEntity> where TEntity
: class
{
    protected readonly AppDbContext _context;
```

```
    // O DbSet é a representação da tabela em memória
    protected readonly DbSet<TEntity> _dbSet;

    public RepositoryBase(AppDbContext context)
    {
        _context = context;
        // O EF Core descobre qual tabela usar baseado no tipo TEntity
        _dbSet = context.Set<TEntity>();
    }
    public async Task<TEntity?> ObterPorId(int id)
    {
        return await _dbSet.FindAsync(id);
    }
    public async Task<IEnumerable<TEntity>> ObterTodos()
    {
        return await _dbSet.AsNoTracking().ToListAsync();
    }
    public async Task Adicionar(TEntity entity)
    {
        await _dbSet.AddAsync(entity);
        // Lembre-se: Não damos SaveChanges aqui! O Unit of Work fará isso.
    }
    public void Remover(TEntity entity)
    {
        _dbSet.Remove(entity);
    }
}
```

Passo 3: Refatorando os Repositórios Específicos

Agora, veja como o MarcaRepository fica limpo. Ele herda tudo do pai e só implementa o que for **exclusivo** dele.

Local: ITB.Infrastructure/Repositories/MarcaRepository.cs

Código C#

```
// 1. Herdamos de RepositoryBase passando a classe concreta "Marca"
public class MarcaRepository : RepositoryBase<Marca>, IMarcaRepository
{
    public MarcaRepository(AppDbContext context) : base(context)
    {
        // O construtor repassa o contexto para a classe pai (base)
    }

    // Não precisamos escrever Adicionar, Atualizar, Remover ou ObterPorId!
    // Eles já existem automaticamente.
    // Aqui colocamos apenas métodos exclusivos, ex: Buscar por Nome
    public async Task<bool> VerificarExistencia(int id) =>
        await _context.Marcas.AnyAsync(m => m.Id == id);

    public async Task<bool> PossuiVeiculosAtivos(int marcaId)
    {
        // O AnyAsync é traduzido para um "IF EXISTS (SELECT 1 ...)" no SQL.
        // É a forma mais rápida de verificar existência sem carregar dados para a
memória.
        return await _context.Veiculos
            .AnyAsync(v => v.MarcaId == marcaId && v.Ativo);

        // DICA: Se você configurou o Global Query Filter (x => x.Ativo) no mapeamento,
        // a checagem "&& v.Ativo" é opcional, pois o EF já filtra automaticamente.!
    }
}
```

Passo 4: Atualizando as Interfaces Específicas

As interfaces também devem herdar da interface base para manter o contrato.

Local: ITB.Domain/Interfaces/IMarcaRepository.cs

Código C#

```
public interface IMarcaRepository : IRepositoryBase<Marca>
{
    // Apenas métodos específicos da Marca
    Task<bool> VerificarExistencia(int id);
    Task<bool> PossuiVeiculosAtivos(int marcaId);
}
```

📚 Resumo da Aula

1. **Definição:** Generics são moldes de código que aceitam tipos variados.
2. **Sintaxe:** Usamos <T> para definir o tipo genérico e where T : ... para impor regras.
3. **Na Prática:** O RepositoryBase<T> eliminou cerca de 80% do código repetido na nossa camada de Infraestrutura.
4. **O Poder do EF Core:** O comando _context.Set<T>() é o segredo para criar repositórios genéricos com Entity Framework.

🛠 Exercício de Fixação

Objetivo: Refatorar o VeiculoRepository e o ModeloRepository.

1. Faça a interface IVeiculoRepository herdar de IRepositoryBase<Veiculo>.
2. Faça a classe VeiculoRepository herdar de RepositoryBase<Veiculo>.
3. **Apague** os métodos Adicionar, Atualizar, Remover, ObterPorId e ObterTodos que estavam escritos manualmente na classe filha.
4. Mantenha apenas os métodos customizados (como ObterPorPlaca ou ObterPorMarcaId).
5. Rode a aplicação e teste se os endpoints continuam funcionando normalmente.